Proteus: Constructing Semantically Equivalent Program Clones using LLMs and Execution Traces

Benita Magdalene Kingsley Prince
School of Computing
Montclair State University
Montclair, USA
0009-0005-0174-3986

Ioannis Agadakos
School of Computing
Montclair State University
Montclair, USA
0000-0001-8610-6954

***Abstract*—Code clone *detection*—deciding whether two programs are semantically equivalent despite syntactic differences—has matured into a rich body of work spanning source- and binary-level analysis. The inverse problem, however, remains largely unexplored: given a program, can we *generate* a functional clone that preserves its observable behavior while presenting a materially different implementation? This capability is valuable for software diversification, dependency modernization (e.g., replacing a vulnerable or slow library with a safer or faster one), and the resilience of critical-infrastructure software. In this paper, we present *Proteus*, an agentic system that, given a source program and a set of requirements, synthesizes a working code clone through the coordinated action of four Large-Language-Model-driven agents: a *library-substitution* agent that swaps third-party dependencies for functionally comparable but more secure or performant alternatives, a *glue-synthesis* agent that adapts the surrounding program to the new dependencies, a *logic-mutation* agent that rewrites first-party application logic into behaviorally equivalent yet textually divergent code, and a *verification* agent that exercises available test cases to confirm equivalence and localize regressions. *Proteus* couples two tiers of verification: a *local* execute-and-repair loop that guards each transformation, and a *global* equivalence oracle that confirms end-to-end equivalence or pinpoints where the clone diverges. We evaluate *Proteus* on twelve programs that estimate factory measurements and control industrial infrastructure, against an unverified single-shot baseline and an ablation without the verification tier. Conventional measures—compilation, test-suite pass rate, oracle confirmation—fail to separate the three configurations, because every subject injects its dependencies and so never executes the migrated code path under test. A blind review of all 108 generated clones by an independent model does separate them: 33% of the clones *Proteus* accepts carry serious behavioral defects, against 54% for the single-shot baseline. *Proteus* accepts far fewer clones, however, and therefore delivers slightly fewer sound ones overall; on this corpus the difference is not statistically significant. The ablation performs indistinguishably from the single-shot baseline, indicating that the verification tier rather than the agent decomposition carries the effect. These early results suggest that agentic, verification-driven mutation is a promising route to automated, trustworthy software diversification, and that evaluating it requires oracles that execute the substituted path directly.**

***Index Terms*—Large Language Models, Agentic AI, Code Clone Generation, Program Transformation, Software Diversity, Industrial Control Systems**

I. Introduction

Software rarely exists in a single, immutable form. The same functionality is routinely re-expressed across languages, libraries, and coding styles as systems are ported, hardened, and modernized. This plurality is not incidental—it is a defensive and engineering asset. *Software diversity* raises the cost of exploitation by denying an attacker a single, uniform target [1], and dependency modernization—replacing an outdated or vulnerable component with a safer or faster one—is a recurring maintenance burden in long-lived deployments. Nowhere is this burden more acute than in *critical-infrastructure* software, such as the code that estimates measurements on a factory floor or drives an industrial control system (ICS), where a component must be replaced or diversified without altering the process it governs. In such settings the central requirement is exacting: the new program must behave *exactly* like the old one, differing only where we intend it to differ.

For over two decades, the research community has studied the recognition side of this equivalence question under the banner of *code clone detection* [2], [3], [4], the task of deciding whether two programs are semantically equivalent despite syntactic differences. Detection underpins vulnerability search, malware triage, plagiarism analysis, and intellectual-property protection, and recent work—including our own prior study of Large-Language-Model (LLM)-assisted binary clone detection [5]—has shown that modern models, when augmented with program-analysis tooling, can reason about semantic equivalence with useful accuracy. The inverse problem, by contrast, has received little attention: rather than *recognizing* that two programs are clones, can we *construct* a clone of a given program on demand?

We argue that automated clone *generation* is both timely and tractable. It is timely because LLMs have demonstrated strong performance across code synthesis, translation, and repair [6], [7], [8], and because agentic architectures now let these models act, observe the results of their actions, and iteratively refine their output [9], [10]. It is tractable because generation, unlike detection, admits a *constructive* correctness check: a candidate clone can simply be executed and compared against the original. The key insight is that an LLM need not produce a correct clone in one shot; it needs only to

produce a candidate that a verification loop can execute, test, and repair until equivalence holds. Importantly, this shifts the burden from the model's fallible one-shot reasoning—a well-documented source of hallucination—onto concrete runtime evidence.

To this end, we present *Proteus*, an agentic system that synthesizes a functional code clone from a given source program and a set of transformation requirements. *Proteus* decomposes the problem into four cooperating, LLM-driven agents, each responsible for a distinct class of change and each guarded by its own execute-and-repair loop. A *library-substitution* agent replaces third-party dependencies (e.g., a cryptographic or numerical library) with functionally comparable alternatives that are more secure or faster; a *glue-synthesis* agent writes the adapter code that reconciles the program with the substituted dependency's interface; a *logic-mutation* agent rewrites *first-party* application logic—not third-party code—into behaviorally equivalent but textually divergent form; and a *verification* agent runs any available test cases to confirm equivalence and to flag inconsistencies introduced by any upstream agent. Above these per-agent loops, *Proteus* applies a global equivalence oracle—our prior binary clone detector [5]—that either confirms the clone is equivalent to the original or localizes the divergence for targeted repair.

We evaluate *Proteus* on software drawn from the industrial-measurement and infrastructure-control domain, where behavioral fidelity is a hard constraint rather than a convenience. Our contributions are:

- We formulate *automated code clone generation* as the constructive inverse of clone detection, and we identify verification-driven, agentic mutation as a practical means of achieving it.
- We propose *Proteus*, a four-agent architecture that separates dependency substitution, glue synthesis, first-party logic mutation, and cross-cutting test-based verification, with a two-tier (local execute-and-repair and global oracle-based) correctness discipline.
- We develop a prototype that reuses our prior binary clone detector as an equivalence oracle, closing the loop between clone generation and clone detection.
- We evaluate *Proteus* on industrial measurement and control software and report clone-generation success, verification cost, and the degree of achieved structural divergence.

The rest of this paper is organized as follows. Section II provides background on code clones, agentic LLMs, and the industrial-control setting. Section III presents the *Proteus* architecture and its two verification tiers. Section IV describes our prototype. Section V reports our evaluation. Section VII surveys related work, and Section VIII concludes.

II. Background

This section establishes the three pillars on which *Proteus* rests: the taxonomy of code clones that defines what it means to generate one, the agentic LLM paradigm that drives each transformation, and the industrial-control setting that motivates our fidelity requirements.

A. Code Clones and the Generation Problem

A *code clone* is a fragment of code that is equivalent to another under some notion of similarity. The literature conventionally distinguishes four clone types [2], [11], ordered by increasing difficulty of recognition:

- **Type I (Identical):** fragments that are textually identical up to whitespace, layout, and comments.
- **Type II (Syntactic):** fragments identical up to renaming of identifiers and literals, preserving syntactic structure and control flow.
- **Type III (Near-miss):** fragments that add, remove, or modify statements while remaining largely similar.
- **Type IV (Semantic):** fragments that implement the same functionality through different syntax or algorithms (e.g., recursion versus iteration).

Detection research treats these types as targets to *recognize*; *Proteus* treats them as targets to *produce*. Specifically, our goal is the deliberate synthesis of *Type IV* clones—programs that are behaviorally equivalent to an original yet structurally divergent enough to resist naive, morphology-based comparison. This framing turns the detector's hardest case into the generator's objective. Importantly, generation enjoys an asymmetry that detection does not: whereas a detector must *infer* equivalence from two fixed artifacts, a generator can *establish* equivalence constructively, by executing a candidate and comparing its behavior against the original. *Proteus* exploits this asymmetry throughout.

B. Agentic Large Language Models

Modern LLMs are strong code models: trained on vast program corpora, they synthesize, translate, and repair code across languages [6], [7], [8]. Used as one-shot oracles, however, they are unreliable—prone to hallucinated APIs, subtle logical errors, and confident-but-wrong output. The *agentic* paradigm mitigates this by embedding the model in a perceive–act–observe loop: the model proposes an action (e.g., a code edit), an external tool executes it, and the resulting evidence (compiler errors, test failures, traces) is fed back for the next iteration [9]. Iterative self-critique and self-repair—*Reflexion* [10], *Self-Refine* [12], and execution-grounded self-debugging [13]—have been shown to substantially improve correctness on program-synthesis and issue-resolution tasks [14]. *Proteus* builds directly on this insight: each of its agents is an LLM wrapped in an execute-and-repair loop, and the model's output is never trusted until concrete runtime evidence confirms it.

C. Industrial Measurement and Control Software

Our motivating domain is software that estimates physical measurements in factories and controls industrial infrastructure. Such code exhibits three properties that make it a demanding, and therefore informative, target for clone generation. First, *behavioral fidelity is a hard constraint*: a diversified

or modernized controller that computes even slightly different set-points is not an acceptable clone but a hazard. Second, these programs frequently depend on *third-party libraries*—numerical, cryptographic, protocol, and I/O components—whose security or performance characteristics are precisely what an operator may wish to improve, making dependency substitution the natural first transformation. Third, they are often accompanied by *test suites and reference traces* arising from safety and certification requirements, which *Proteus* reuses as ready-made verification evidence. In other words, the same rigor that makes this domain unforgiving also supplies the oracles that make automated clone generation checkable.

III. Approach

A. Overview

Proteus takes two inputs—a working *source program* and a set of *requirements* that state which properties the clone must change (e.g., "replace the TLS library with a memory-safe alternative," "prefer faster numerical routines," or "obscure the proprietary estimation logic")—and produces a *clone*: a program that reproduces the observable behavior of the original while differing from it in the intended ways. Figure 1 presents the architecture. The transformation is not performed in a single pass; instead, *Proteus* routes the program through a pipeline of four specialized agents, each of which owns a distinct class of change. The key design principle is *separation of concerns under verification*: every agent mutates only the code it is responsible for, and no agent's output advances to the next stage until it has been shown to execute correctly. The core engine of each agent is an LLM prompted with the current program state, the agent's specific objective, and—crucially—the concrete evidence (errors, test results) from its most recent attempt.

B. The Transformation Agents

1. Library-Substitution Agent (LSA). The *LSA* is responsible for third-party dependencies. Given the requirements, it identifies candidate libraries whose functionality subsumes that of an incumbent dependency but whose security or performance profile is superior (e.g., a memory-safe parser, a constant-time cryptographic primitive, or a vectorized numerical kernel). It selects a replacement and records the interface delta—the functions, types, and invariants that differ between the incumbent and the substitute. Note that the *LSA* does not itself rewrite the surrounding program; it establishes *what* must change so that downstream agents can act.

2. Glue-Synthesis Agent (GSA). Substituting a dependency is rarely a drop-in replacement: signatures differ, initialization differs, and error-handling conventions differ. The *GSA* synthesizes the *glue code*—adapters, shims, and initialization logic—that reconciles the program with the substitute's interface, using the interface delta produced by the *LSA*. In other words, the *GSA* makes the new dependency *fit* without disturbing the application logic that consumes it.

3. Logic-Mutation Agent (LMA). The *LMA* targets *first-party* application logic—and only first-party logic. It rewrites the program's own routines into behaviorally equivalent but textually and structurally divergent form: it may restructure control flow, re-express algorithms (e.g., iterative for recursive), rename and re-decompose functions, and alter data representations, while preserving observable behavior. We deliberately exclude third-party logic from the *LMA*'s scope: mutating a well-tested library's internals introduces risk without benefit, whereas mutating first-party logic is precisely what yields a Type IV clone of the code that matters. The key insight is that dependency substitution and logic mutation are *orthogonal* axes of divergence, and separating them keeps each transformation small, local, and checkable.

4. Verification and Consistency Agent (VCA). The *VCA* is a cross-cutting agent that does not mutate code but interrogates it. It runs any available test cases—unit tests, integration tests, and reference input/output traces shipped with the subject program—against the evolving clone, both to confirm behavioral equivalence with the original and to detect *ongoing inconsistencies*: regressions that reveal an upstream agent (the *LSA*, *GSA*, or *LMA*) has made an incorrect change. When the *VCA* observes a failure, it attributes the regression to the responsible stage and triggers re-entry into that stage's repair loop, rather than blindly retrying the whole pipeline. As such, the *VCA* acts as a continuous quality gate that turns the pipeline from a fragile straight-line process into a self-correcting one.

C. Two Tiers of Verification

Proteus enforces correctness at two granularities, and this two-tier discipline is what makes agentic mutation trustworthy rather than merely plausible.

Local verification (execute-and-repair). Each transformation agent is wrapped in its own loop. After the agent proposes a change, *Proteus executes* the resulting program (compiling and running it, or running the affected component in isolation). If execution fails—a compile error, a crash, an unmet assertion—the failure evidence is returned to the same agent, which repairs its output and retries. Only when the intermediate program executes cleanly does control advance to the next agent. This localizes errors at the point they are introduced and prevents them from compounding downstream.

Global verification (equivalence oracle). Passing local execution establishes that the clone *runs*; it does not establish that it is *equivalent* to the original. For end-to-end equivalence, *Proteus* invokes a global oracle in two complementary forms. First, the *VCA*'s test-based checks compare clone and original behavior over the available test corpus. Second, *Proteus* reuses our prior binary clone detector [5], which compares the compiled original and clone using LLM reasoning over emulator-generated execution traces, to *confirm* semantic equivalence or to *localize* the divergence when the two disagree. When the oracle reports a divergence, its localization is fed back as a targeted repair signal—identifying which routine or component drifted—so that the responsible agent can correct it.

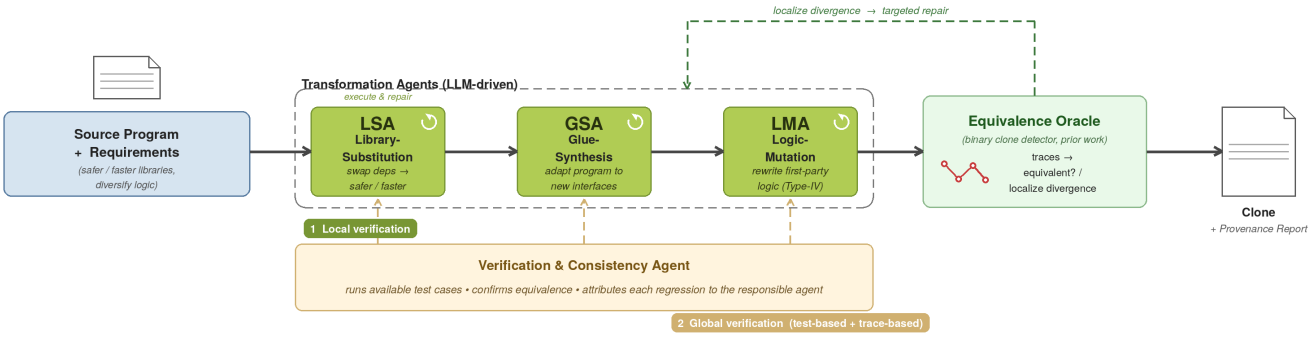


Fig. 1: Architecture of *Proteus*. A source program and a set of requirements enter a pipeline of four LLM-driven agents: the *Library-Substitution Agent* selects safer or faster replacements for third-party dependencies, the *Glue-Synthesis Agent* adapts the program to the new interfaces, and the *Logic-Mutation Agent* rewrites first-party logic into equivalent but divergent form. Each agent is wrapped in a local execute-and-repair loop (dashed self-edges). Spanning the pipeline, the *Verification and Consistency Agent* runs available test cases to confirm equivalence and to attribute regressions to the responsible stage, while a global equivalence oracle—our prior binary clone detector—certifies end-to-end equivalence or localizes divergence. *Takeaway:* correctness is enforced at two tiers—locally per transformation and globally over the finished clone—so that no unverified change reaches the output.

This closes the loop between generation and detection: the same machinery that recognizes clones now certifies the ones *Proteus* constructs.

D. Putting It Together

A single run therefore proceeds as follows. The *LSA* selects substitutions; the *GSA* integrates them; the *LMA* diversifies first-party logic; and throughout, each stage's execute-and-repair loop guarantees the intermediate program runs, while the *VCA* continuously checks equivalence and reassigns blame on regression. When all agents have completed and the global oracle confirms equivalence, *Proteus* emits the clone together with a provenance report describing every substitution and mutation applied—an audit trail that is itself valuable in the certification-heavy settings we target.

IV. Implementation

We implement *Proteus* as an end-to-end pipeline in Python. Each of the four agents is realized as an LLM-backed module driven by `claude-sonnet-5` at temperature 0.7, orchestrated as a LangGraph state machine through a common controller that manages prompting, tool invocation, and the per-agent execute-and-repair loop. Agents communicate through a shared, structured project state—the current source tree, the interface delta emitted by the *LSA*, and the running verification log—serialized as JSON so that any agent can inspect the artifacts produced upstream.

Execution and sandboxing. Local verification requires actually running untrusted, machine-generated code. *Proteus* stages each candidate clone into an isolated temporary working directory and executes it with the host interpreter under a wall-clock ceiling, capturing compiler diagnostics, exit status, standard output, and any assertion failures. This evidence is returned verbatim to the responsible agent as the repair signal, grounding each retry in concrete runtime feedback rather than speculation. We note the isolation boundary as a limitation rather than a strength: it is filesystem-level, not container-level, so a clone that reached the network or the wider filesystem would not be contained. The corpus is our own and the generated code is inspected, but deployment on untrusted subjects would require a container or VM boundary.

Test harness. The *VCA* discovers and drives whatever test assets accompany a subject program—unit and integration tests as well as recorded reference input/output traces—through a uniform adapter, so that heterogeneous test formats present a single pass/fail/attribution interface to the pipeline.

Equivalence oracle. For global verification we use a four-layer source-level oracle, applied in increasing order of cost. The first layer validates that the clone's imports resolve and that the migrated-away distribution is genuinely absent. The second replays recorded golden vectors—inputs and expected outputs captured from the pristine original—at zero numerical tolerance. The third checks program invariants that must hold irrespective of implementation, such as monotonicity of a control output or the reachability of every declared state. The fourth runs the subject's own `pytest` suite against the clone. A verdict of equivalence requires all four to pass, and the first layer to fail identifies the responsible agent directly.

Evaluation exposed a systematic blind spot in this arrangement, and we added a fifth layer in response. Every subject in the corpus dependency-injects its I/O client so that its tests can run without live hardware, which means the factory function that actually constructs the substituted library is never

executed by the suite. The fifth layer calls that factory directly against an unroutable address, and classifies the resulting exception: a connection error is expected and benign, whereas a `TypeError`, `AttributeError` or `ValueError` indicates the migrated call is malformed in a way no absent server could explain. This layer caught defects on clones that had passed all four preceding layers.

Toward binary-level confirmation. The architecture in Section III anticipates reusing our prior binary clone detector [5] as the global oracle, disassembling the compiled original and clone, generating emulator execution traces, and prompting an LLM to decide semantic equivalence and justify its verdict. That integration is not exercised in the results reported here; the equivalence-confirmation figures in Section V come from the source-level oracle described above. Coupling the two—so that the detector's localization of divergence is parsed into a targeted repair signal for the responsible agent—remains the most direct extension of this work.

a) Subject Programs: Our evaluation corpus consists of twelve Python programs from the industrial measurement and control domain, organized into four categories of increasing transformation demand and ranging in complexity from a single self-contained module to multi-file packages with internal dependencies (Section V, Table I). The programs were authored for this study rather than harvested from public repositories, a trade we revisit in Section V: it costs external validity, and it is what makes the per-program golden vectors and invariants the oracle consumes available at all. Every subject ships a `pytest` suite and recorded input/output traces, which serve both as the *VCA*'s test corpus and as ground truth for the equivalence oracle.

V. Evaluation

Our evaluation asks three questions: (RQ1) *Effectiveness*—can *Proteus* produce clones that are behaviorally equivalent to their originals? (RQ2) *Divergence*—are the generated clones structurally different enough to constitute genuine diversification rather than cosmetic edits? (RQ3) *Cost*—how much verification effort, in repair iterations and tokens, does clone generation require? We answer them on the industrial measurement and control corpus described in Section IV.

A. Evaluation Corpus

Table I lists our subject programs. The corpus is designed around two axes that jointly isolate the contribution of each transformation agent. The first axis is *transformation demand*, expressed as four categories: Category A requires only a drop-in dependency swap (exercising the *LSA*); Category B additionally requires glue code to reconcile a new library's interface (exercising the *GSA*); Category C additionally requires rewriting first-party logic (exercising the *LMA*), for example when the substitution removes a third-party control library and thereby turns its behavior into first-party code that must be diversified; and Category D combines all three transformations within a single realistic program—an API-incompatible library swap, its integration glue, and substantial first-party control or estimation logic to diversify—so that the agents must cooperate end-to-end. The second axis is *structural complexity*, ranging from a single self-contained module to multi-file packages with internal dependencies. All subjects are Python programs drawn from the industrial measurement and control domain, and each ships a `pytest` suite together with recorded input/output traces that serve as both the *VCA*'s test corpus and the equivalence oracle's ground truth. In other words, the corpus supplies, per program, exactly the verification evidence *Proteus* consumes.

Two subjects, `modbus_meter_reader` (B2) and `boiler_control_loop` (D1), declare a substitution that cannot be satisfied as stated: both originals speak Modbus TCP via `ModbusTcpClient(host, port=502)`, whereas the declared target `minimalmodbus` is serial-only and exposes no TCP entry point. We discovered this during evaluation rather than by design. Rather than repair the corpus we retained both as *negative controls*, and they proved informative: across the nine `boiler_control_loop` runs, five independently selected `pyModbusTCP` over the declared target, and the single run that did adopt `minimalmodbus` was caught passing a TCP port number as a Modbus slave address.

B. Metrics

We define the following metrics before reporting results, so that success criteria are fixed independently of the outcomes.

- **Generation success rate:** the fraction of subject programs for which *Proteus* emits a clone that compiles and passes the full available test suite.
- **Equivalence-confirmation rate:** of the clones that pass local testing, the fraction the equivalence oracle certifies as semantically equivalent to the original. For the results reported here the oracle is the four-layer source-level checker described in Section IV: import validation, recorded golden vectors at zero tolerance, program invariants, and the subject's own test suite. It is reported over Categories C and D only—the six subjects that ship golden vectors and invariants, giving eighteen runs per configuration. Categories A and B have no equivalence probes; scoring them as passes would present a test run as an equivalence proof.
- **Migration success:** an accepted clone. Stricter than generation success and used as the acceptance signal throughout: the clone compiles, passes its full suite and the oracle, the source genuinely changed, and the original dependency is verifiably absent. The last two conditions are necessary because every subject passes its own tests *before* migration, so generation success alone cannot separate a real migration from an untouched file.
- **Structural divergence:** the degree to which the clone differs from the original, reported as the unweighted mean of two rename-insensitive measures taken between the delivered clone and the pristine original over the concatenation of all first-party files: the Ratcliff/Obershelp similarity of the pre-order AST *node-type* sequence, which ignores

Program	Files	Dependency substitution (from → to)	Description — ICS function	Validity aspect a correct clone must satisfy
Category A — Drop-in library substitution (exercises the LSA only; no glue or logic change)				
`crc_telemetry`	1	`zlib.crc32` → `fastcrc`	Validates the integrity of RTU/Modbus telemetry frames via CRC-32.	Bit-identical checksums after a drop-in swap; behavioral equivalence with *no* glue or logic change.
`sensor_json_logger`	1	`json` → `ujson`	Serializes tag/value sensor samples to JSON audit-log lines.	Byte-identical serialized records (key order, numeric formatting); the LSA does not perturb output.
`iso_ts_parser`	1	`datetime` → `ciso8601`	Parses ISO-8601 timestamps from event and alarm logs.	Identical parsed values across edge cases (time zones, fractional seconds) and equivalent error handling.
Category B — Substitution requiring glue code (exercises the LSA + GSA; new API, same behavior)				
`scada_poller`	1	`requests` → `httpx`	Polls a SCADA REST gateway for tag values on a fixed interval.	The GSA reconciles session/response/timeout API deltas; equivalent retrieved values and error-path behavior.
`modbus_meter_reader`	2	`pymodbus` → `minimalmodbus`	Reads holding registers from a power meter and assembles a measurement record.	The GSA bridges register-read and word-decoding API differences; identical decoded measurements and mapped exceptions.
`mqtt_telemetry_bridge`	3	`paho-mqtt` → `aiomqtt`	Publishes sensor telemetry to an MQTT broker with QoS and reconnect.	The GSA adapts the sync-callback model to async glue; equivalent topics, payloads, and QoS/reconnect semantics.
Category C — Substitution + first-party logic mutation (exercises the LSA + GSA + LMA; multi-file, internal dependencies)				
`pid_pump_controller`	3	`simple-pid` → in-house controller	PID loop regulating pump speed to a tank-level setpoint.	Removing the control library makes the PID first-party; the LMA rewrites the update into divergent-but-equivalent form; the output trajectory matches within tolerance.
`tank_level_fsm`	3	`transitions` → hand-rolled FSM	Finite-state controller for a fill/drain process with interlocks and alarms.	The LMA re-encodes the state machine (e.g., table-dispatch vs. class hierarchy) while preserving every transition, interlock, and alarm output over recorded event sequences.
`anomaly_threshold_engine`	4	`smtplib` notifier → `apprise`	Streaming out-of-band detector with hysteresis, plus config loader and operator notifier.	Full pipeline: LSA+GSA swap the notifier, the LMA rewrites the hysteresis/threshold logic; identical flag/alarm sequence over recorded telemetry and complete provenance.
Category D — Combined substitution + glue + logic mutation (exercises the LSA + GSA + LMA together in one program)				
`boiler_control_loop`	3	`pymodbus` → `minimalmodbus`	Modbus-driven boiler supervisor: reads temperature and pressure, runs PI control with an over-pressure interlock, and actuates the valve.	All three at once: glue for the Modbus API delta (GSA) and an LMA rewrite of the PI-control and interlock logic; identical valve commands and trip behavior over recorded traces.
`pump_station_scheduler`	3	`requests` → `httpx`	SCADA-driven pump station: polls demand and per-pump runtimes, computes lead-lag staging with runtime balancing, and issues pump commands.	All three: HTTP glue (GSA) and an LMA rewrite of the staging/rotation logic; identical pump on/off plan over recorded demand and runtime sequences.
`telemetry_gateway`	3	`paho-mqtt` → `aiomqtt`	Telemetry gateway: reads raw meter counts, estimates engineering units with smoothing and rate-of-change anomaly flags, and publishes to MQTT.	All three: async MQTT publish glue (GSA) and an LMA rewrite of the estimation/anomaly logic; identical value/anomaly stream and topics.

TABLE I: The *Proteus* evaluation corpus: twelve Python industrial measurement and control programs spanning four categories of increasing transformation demand and rising complexity (single module → multi-file package with internal dependencies). Every subject ships a `pytest` suite and recorded input/output traces that serve both as the *VCA*'s test corpus and as ground truth for the equivalence oracle. The requirement given to *Proteus* is the listed substitution for all categories, plus "diversify all first-party logic" for Categories C and D. *Takeaway:* the categories isolate the contribution of each transformation agent—Category A stresses substitution alone, Category B adds interface glue, Category C additionally demands behavior-preserving logic mutation, and Category D combines all three transformations within a single realistic program.

identifier and literal choices, and a weighted Jaccard distance over a depth-tagged histogram of branch constructs, which captures reshaped control flow. The composite lies on $[0, 1]$, higher meaning more divergent. We report a rename-insensitive measure deliberately: token-level edit distance rewards renaming every identifier, which is exactly the cosmetic change a Type II detector defeats, and a diversification result that a detector defeats is not

a result.

- **Repair cost:** the number of local execute-and-repair iterations per agent, and the number of *VCA*-triggered stage re-entries, required to reach an accepted clone.
- **Token and monetary cost:** total tokens and estimated USD per generated clone.
- **Defect rate among accepted clones:** of the clones a configuration *itself accepted*, the fraction an independent blind reviewer finds to carry BROKEN or MAJOR behavioral defects (Section V-D1). This measures the precision of a configuration's own success signal, which none of the automated measures above can assess.

C. Experimental Setup

We run *Proteus* with `claude-sonnet-5` at temperature 0.7, three seeds per subject, a wall-clock ceiling of 180 s per LLM request, and a cap of three *VCA*-triggered stage re-entries per run, and record for each run whether generation succeeded, the metrics above, and the provenance report of applied substitutions and mutations. Temperature is deliberately not zero: the protocol calls for three runs per subject with different seeds so that variance can be reported, and at temperature zero those runs would be near-identical.

Every subject receives the same requirement: remove the dependency named in its `requirements.txt` while preserving observable behavior, and additionally, for Categories C and D, diversify all first-party logic. Crucially, **the replacement library is never named**—not to *Proteus* and not to the baseline. Both must select a target themselves. An earlier protocol supplied the corpus's declared target to *Proteus* while the baseline chose freely; that made the comparison asymmetric and, on the two negative controls, forced *Proteus* toward a library that cannot work.

a) Baselines: To isolate the contribution of *Proteus*'s agentic, verification-driven design, we compare against two configurations. *Baseline 1 (single-shot LLM)* issues one prompt to the same model, naming only the dependency to be removed, with no agents, no tools and no verification loop; the reply is parsed for code blocks and used as-is. We verified by inspection and by telemetry that no tool loop is involved: exactly one LLM call on all 36 runs, averaging 1,471 input tokens, against up to 14 calls and 20,749 input tokens per run for full *Proteus*. *Baseline 2 (*Proteus *without verification)* is an ablation with the *VCA* and the global oracle disabled, leaving only the per-agent local execute-and-repair loop. These baselines let us attribute effectiveness gains to the two-tier verification discipline rather than to the underlying model.

D. Results

Table II reports the conventional measures. They do not separate the three configurations: generation success is 30/36, 33/36 and 29/36, and equivalence confirmation on Categories C and D is 16/18, 17/18 and 16/18. On these numbers alone the ablation is the strongest configuration and full *Proteus* the weakest, which is the opposite of what the two-tier discipline predicts.

Configuration	Gen. Success	Equiv. Conf.	Divergence	Repair Iters	Cost (USD)
Single-shot LLM (B1)	30/36	16/18	0.141	0	$0.79
Proteus w/o verification (B2)	33/36	17/18	0.127	0	$3.87
Proteus (full)	29/36	16/18	0.154	6	$4.91

TABLE II: Clone-generation results on the industrial measurement and control corpus. Twelve programs $\times$ three seeds $=$ 36 runs per configuration; equivalence confirmation is over Categories C and D only (18 runs per configuration). Repair iterations are *VCA*-triggered stage re-entries, summed over all runs. *Takeaway:* the conventional measures do not distinguish the three configurations—the ablation scores highest on two of them—which motivates the independent review reported in Table III.

Configuration	BROKEN	MAJOR	MINOR	CLEAN	NO-OP	Clean rate
Single-shot LLM (B1)	2	16	12	5	0	5/35
Proteus w/o verification (B2)	1	8	1	15	11	15/25
Proteus (full)	5	8	4	16	3	16/33

TABLE III: Blind review of all 108 generated clones by an independent, stronger model with every system-identifying marker removed. The clean rate excludes NO-OP clones, which attempted nothing. One single-shot run has no verdict—the reviewer returned an empty completion on four attempts with exponential backoff—so that configuration's denominator is 35.

The measures are not wrong; they answer a narrower question than they appear to. Every subject's tests pass before migration, and every subject injects its I/O dependency, so the factory code that performs the substitution is frequently never executed by the suite. A clone can therefore compile, pass its full suite and satisfy the oracle while calling the new library incorrectly on the one path the tests do not reach. Compilation discriminates nothing at all: all three configurations compile on 36 of 36 runs.

1) Blind Review of Every Clone: To measure what those instruments cannot, we submitted all 108 generated clones to a blind LLM review. Each clone was shown alongside its original to `claude-opus-5`—deliberately a different and stronger model than the `claude-sonnet-5` that generated it, so that no model grades its own output—and asked only for concrete behavioral differences, each naming a file, quoting a line and stating the runtime consequence. Style, structure and library choice are excluded explicitly. Verdicts are BROKEN (fails on ordinary input), MAJOR (wrong behavior in a case the original handled), or MINOR (a real difference confined to an edge case). All system-identifying markers were renamed or removed before review, and the batch aborts rather than continuing if any marker survives, so the reviewer never learns which configuration produced a clone. NO-OP denotes a clone byte-identical to its original: trivially faithful because nothing was attempted, and therefore excluded from the clean rate rather than counted as a success.

The review separates the configurations where the conventional measures do not (Table III, Table IV, Fig. 2). Of the clones each configuration itself accepted, the fraction carrying

Configuration	Clones accepted	With serious defects	Defect rate
Single-shot LLM (B1)	28/36	15	**54%**
Proteus w/o verification (B2)	11/36	6	**55%**
Proteus (full)	15/36	5	**33%**

TABLE IV: Precision of each configuration's own success signal. Of the clones a configuration certified as an accepted migration, the fraction the blind reviewer found to carry BROKEN or MAJOR defects. Accepting more clones and accepting sound clones are different capabilities, and on this corpus they trade off against one another.

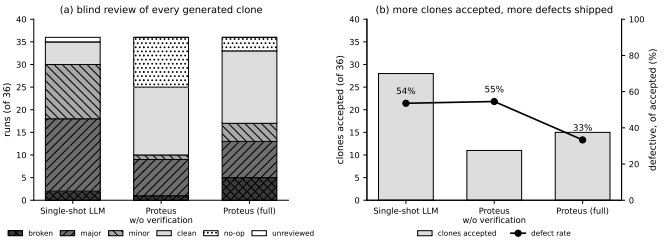


Fig. 2: (a) Every run classified by the blind reviewer. (b) The trade-off: the configuration accepting the most clones ships the highest fraction of defective ones. Read either panel alone and the ranking inverts, which is why both are shown.

BROKEN or MAJOR defects is 54% for the single-shot baseline, 55% for the ablation, and 33% for full *Proteus*.

Two observations follow, and they point in opposite directions.

First, **the ablation performs like the single-shot baseline**: 55% against 54%. Removing the verification tier leaves an agent architecture no more reliable than a single prompt. Whatever *Proteus* contributes is attributable to verification rather than to decomposition into cooperating agents.

Second, **full *Proteus* does not deliver more sound clones than one prompt**. It yields 10 defect-free accepted clones per 36 runs against the baseline's 12. It accepts far fewer clones (15 against 28) and is right more often when it does accept—a precision gain paid for in recall. On a corpus where a wrong clone is worse than no clone, that is the trade we would choose, but it is a trade and not a dominance result.

Cost. The precision is not free. Full *Proteus* consumed 925K tokens and 140 minutes across 36 runs against the baseline's 95K tokens and 6 minutes: roughly a sixfold increase in tokens and a twentyfold increase in wall time, for $4.91 against $0.79.

Statistical caution. With twelve subjects these differences are not statistically significant. Fisher's exact test gives $p = 0.34$ for the single-shot baseline against full *Proteus* on defect rate, and $p = 1.00$ against the ablation; the 95% confidence intervals overlap substantially (54% [36–70], 55% [28–79], 33% [15–58]). We report the direction and its mechanism, not a demonstrated effect. Separating 33% from 54% at $p < 0.05$ would require roughly three to four times this corpus.

E. Analysis

Which stage dominates repair cost. Full *Proteus* triggered six *VCA* stage re-entries across 36 runs, all routed to the *GSA*, and no run exhausted its repair budget. Repair cost is therefore not the binding constraint on this corpus; acceptance is.

Which requirement classes are hardest. The review's findings fall into a small number of recurring shapes, and the two systems fail differently. The single-shot baseline produces 80 findings dominated by *fidelity* failures: 34 concern a changed exception type or message, and 10 concern a client or connection never closed. Its wiring is correct and its semantics drift. *Proteus* produces 45 findings dominated by *integration* failures: arguments accepted by an adapter and then never used (7 against the baseline's 2), adapters generated but not reached (5 against 2), and errors collapsed into a single generic exception (4 against 1). Its adapters are structurally sound—generating a wrapper naturally preserves lifecycle and error translation, which is why it has one unclosed resource against the baseline's ten—and it loses detail at the call sites.

Characteristic failure modes. In the ablation's `boiler_control_loop` clone, an adapter catches `(ModbusError, Exception)` and converts every failure—connection loss and programming errors such as `TypeError` alike—into a generic `IOError`. On a boiler supervisor an adapter bug becomes indistinguishable from a genuine Modbus fault. The ablation accepted this clone; full *Proteus* rejected its counterpart at the *VCA* on the same two seeds. In full *Proteus*'s own `boiler_control_loop` clone, the generated wrappers accept the per-call slave identifier and never use it, so an injected client configured for a different device would be actuated instead. Neither defect is reachable by the test suite, because the suite injects a fake client that ignores the same argument.

What the global oracle catches. Two *Proteus* clones import the migrated-to library while still calling the migrated-from one, which was never imported—a guaranteed `NameError` on the factory path. Both were rejected by the pipeline, at the *LMA* and the *VCA* respectively, and the blind reviewer independently rated both BROKEN. The system's own verdict and an independent reviewer's agree on precisely the clones the test suite passes.

a) Lessons Learned: Three findings we did not anticipate.

Test suites are the wrong instrument for this task. Dependency injection is good practice, and it makes the migrated code path untestable by the very suite that certifies it. Any future benchmark of clone generation needs an oracle that executes the substituted path directly, which is why we added factory execution as a fifth oracle layer mid-study.

Separating substitution from mutation keeps repairs local, but the seam leaks. The *LSA*/*GSA* split behaves as intended—repairs stay within one agent and no run exhausted its budget—yet the residual failure mode lives precisely at the boundary, where an adapter is generated correctly and then not fully wired to the call sites it was written for. Orthogonal concerns still require a shared contract at their interface.

One third of the corpus is unsolved by every configuration. Four subjects—the MQTT bridge, the PID controller, the threshold engine and the telemetry gateway—produced zero defect-free accepted clones across all 27 runs. Category C stands at 1 of 27 across all configurations: the requirement to replace a dependency with first-party code is, on this evidence, not differentially solved but essentially unsolved. *Proteus*'s resolver selects among libraries and has no mechanism to synthesize a replacement implementation; the single-shot baseline does write a PID controller and a state machine by hand, but only one such clone survived review.

VI. Discussion and Limitations

We frame *Proteus*'s limitations precisely and bound their scope.

Oracle-bounded correctness. *Proteus* certifies equivalence only up to the strength of its verification evidence. Behavioral equivalence is established over the available test corpus and the oracle's recorded vectors and invariants; it is not a formal proof. Consequently, an equivalence-confirmed clone is guaranteed equivalent *on the exercised behaviors*, and residual divergence can persist on inputs neither the tests nor the oracle probes reach. This bound is well-understood and directly tied to test coverage: richer suites and traces tighten it, and the certification-heavy domains we target tend to supply exactly such assets. The blind review reported above is our attempt to probe outside that bound, and it found defects on every configuration that the automated evidence had accepted.

Scope of mutation. By construction the *LMA* mutates only first-party logic, so the divergence *Proteus* achieves is bounded by the ratio of first-party to third-party code in a subject program; dependency-dominated programs diversify primarily through substitution. A sharper limit surfaced in evaluation: the *LSA* selects *among* libraries and cannot synthesize a first-party replacement for one, so on the two Category C subjects whose requirement is precisely that, it either substitutes a different library or retains the incumbent.

Corpus scale and provenance. The twelve subjects are our own, authored for this study rather than harvested from public repositories. That costs external validity, and it is why the comparisons in Section V are reported as directions rather than established effects. It is also what makes per-program golden vectors and invariants available at all; without those the equivalence question cannot be posed at the granularity this study requires.

Dual use. The ability to generate equivalent-but-divergent code is defensively valuable for diversification, but the same capability could aid evasion of similarity-based detection. We discuss this tension and the role of provenance reporting in Section VIII.

VII. Related Work

Proteus sits at the confluence of four lines of research: clone and similarity detection, LLM-based code synthesis and translation, agentic self-repair, and automated software transformation. We organize the fifteen most closely related works thematically and state, for each cluster, the specific and bounded way *Proteus* differs.

A. Code Clone and Similarity Detection

The recognition side of code equivalence is mature. Roy, Cordy, and Koschke [2] provide the canonical comparison of clone detection techniques and the Type I–IV taxonomy we adopt as our generation target. *SourcererCC* [3] scales token-based near-miss (Type III) detection to hundreds of millions of lines, demonstrating that similarity can be recognized at ecosystem scale. At the binary level, the survey of Haq and Caballero [4] systematizes two decades of binary code similarity, and our own prior work builds an LLM-plus-emulator detector on this foundation [5]. Unlike all of these, which *decide* whether two given artifacts are clones, *Proteus constructs* a clone of a given program; we then repurpose a detector [5] as a certification oracle, using recognition in service of generation.

B. LLMs for Code Synthesis and Translation

Large language models trained on code—exemplified by Codex [6], competition-grade AlphaCode [7], and the program-synthesis benchmarks of Austin et al. [15]—establish that models can generate correct programs from specification. Most relevant to us, *TransCoder* [8] performs unsupervised translation between programming languages, producing behavior-preserving but syntactically distinct code—precisely the equivalence-under-divergence property *Proteus* seeks. In contrast to this cluster, which treats generation as a *single* model producing output judged against a fixed benchmark, *Proteus* embeds generation in a multi-agent pipeline with per-stage execution feedback and an external equivalence oracle, and it targets whole-program diversification under an operator-specified requirement set rather than function-level synthesis.

C. Agentic Reasoning and Self-Repair

Proteus's reliability derives from iterative, evidence-grounded refinement. *ReAct* [9] interleaves reasoning with tool-use actions; *Reflexion* [10] and *Self-Refine* [12] show that verbal self-critique improves successive attempts; execution-grounded self-debugging [13] repairs code from runtime feedback; and *SWE-bench* [14] demonstrates agentic resolution of real software issues. *Proteus* adopts this execute-observe-repair discipline, but differs in two respects: it assigns *distinct* agents to orthogonal transformation concerns (dependency substitution, glue synthesis, first-party logic mutation), and it adds a dedicated cross-cutting verification agent that attributes regressions to the responsible stage—turning generic self-repair into blame-localized repair across a pipeline.

D. Software Diversity, Obfuscation, and Automated Repair

Finally, *Proteus*'s goal—behavior-preserving structural change—has deep roots in program-transformation research. Larsen et al. [1] systematize automated software diversity as a defense that denies attackers a uniform target, and Collberg et al. [16] catalog obfuscating transformations that alter form

while preserving semantics. Automated program repair, exemplified by *GenProg* [17], similarly mutates programs under a test-based acceptance criterion. However, these approaches rely on fixed, hand-engineered transformation rules or search over edits; *Proteus* instead drives semantics-preserving transformation with LLM agents that reason about *intent* (e.g., "choose a safer library"), and it certifies the result with a semantic equivalence oracle rather than a test suite alone.

VIII. Conclusions

We introduced automated code clone *generation* as the constructive inverse of clone detection, and we presented *Proteus*, an agentic system that synthesizes a behaviorally faithful but structurally divergent clone of a given program. *Proteus* decomposes the task across four cooperating LLM-driven agents—library substitution, glue synthesis, first-party logic mutation, and cross-cutting verification—and enforces correctness at two tiers: a local execute-and-repair loop that guards every transformation, and a global equivalence oracle that certifies the finished clone or localizes its divergence. The key insight is that generation, unlike detection, permits constructive verification: an LLM need not be correct in one shot, because a disciplined loop can execute, test, and repair its output until equivalence holds.

On software that estimates factory measurements and controls industrial infrastructure—a domain where behavioral fidelity is a hard constraint—*Proteus* generates equivalence-confirmed clones for 16 of 18 Category C and D runs at a median of zero repair iterations, and reduces the share of accepted clones carrying serious behavioral defects from 54% to 33% relative to an unverified single-shot baseline. Two caveats belong with that number. The difference is in the predicted direction but this corpus is too small to establish it ($p = 0.34$, Fisher's exact). And *Proteus* accepts far fewer clones than the baseline, so it delivers slightly fewer sound ones overall: the gain is in the precision of its acceptance signal, not in throughput. A configuration that trades recall for precision is the right default in a domain where a plausible-looking but wrong clone is worse than no clone, but it is a trade rather than a dominance result.

The ablation sharpens the attribution. *Proteus* with its verification tier disabled behaves almost identically to the single-shot baseline—55% against 54%—which indicates that the agent decomposition on its own contributes little, and that what separates the full system is the verification discipline rather than the division of labor among agents.

Three directions follow directly. First, we will strengthen the oracle from test- and trace-based checking toward partial formal equivalence on the safety-critical core of control logic, tightening the coverage-bounded guarantee discussed in Section V; our evaluation found defects that every automated layer accepted, so the bound is not merely theoretical. Second, the residual failure mode is concentrated at the seam between agents—adapters generated correctly and then incompletely wired to their call sites—which is a program-analysis problem rather than a generation problem, and suggests pairing each agent with a static check on the contract it is expected to honor. Third, because the capacity to produce equivalent-but-divergent code is inherently dual-use—defensively valuable for diversification, yet potentially useful for evading similarity-based detection—we will pair every generated clone with the machine-checkable provenance report *Proteus* already emits, so that each transformation is attributable and auditable. Recognizing and generating clones are two faces of the same problem; coupling them, as *Proteus* does, turns program equivalence from a question we merely answer into a property we can deliberately engineer.

References

[1] P. Larsen, A. Homescu, S. Brunthaler, and M. Franz, "SoK: Automated software diversity," in *2014 IEEE Symposium on Security and Privacy (SP)*, IEEE, 2014, pp. 276–291.

[2] C. K. Roy, J. R. Cordy, and R. Koschke, "Comparison and evaluation of code clone detection techniques and tools: A qualitative approach," *Science of Computer Programming*, vol. 74, no. 7, pp. 470–495, 2009.

[3] H. Sajnani, V. Saini, J. Svajlenko, C. K. Roy, and C. V. Lopes, "SourcererCC: Scaling code clone detection to big-code," in *Proceedings of the 38th International Conference on Software Engineering (ICSE)*, 2016, pp. 1157–1168.

[4] I. U. Haq and J. Caballero, "A survey of binary code similarity," *ACM Comput. Surv.*, vol. 54, no. 3, Apr. 2021, ISSN: 0360-0300. DOI: 10.1145/3446371 [Online]. Available: https://doi.org/10.1145/3446371

[5] I. Agadakos, *Leveraging LLMs and emulator-based execution traces for binary clone detection*, Companion work; update with final venue/citation, 2025.

[6] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, et al., "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.

[7] Y. Li, D. Choi, J. Chung, N. Kushman, J. Schrittwieser, et al., "Competition-level code generation with AlphaCode," *Science*, vol. 378, no. 6624, pp. 1092–1097, 2022.

[8] B. Roziere, M.-A. Lachaux, L. Chanussot, and G. Lample, "Unsupervised translation of programming languages," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 20 601–20 611.

[9] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations (ICLR)*, 2023.

[10] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[11] S. H. Ding, B. C. Fung, and P. Charland, "Asm2vec: Boosting static representation robustness for binary clone search against code obfuscation and compiler

optimization," in *2019 IEEE Symposium on Security and Privacy (SP)*, IEEE, 2019, pp. 472–489.

[12] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, et al., "Self-Refine: Iterative refinement with self-feedback," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[13] X. Chen, M. Lin, N. Schärli, and D. Zhou, "Teaching large language models to self-debug," in *arXiv preprint arXiv:2304.05128*, 2023.

[14] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world GitHub issues?" In *International Conference on Learning Representations (ICLR)*, 2024.

[15] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, et al., "Program synthesis with large language models," *arXiv preprint arXiv:2108.07732*, 2021.

[16] C. Collberg, C. Thomborson, and D. Low, "A taxonomy of obfuscating transformations," Department of Computer Science, The University of Auckland, Tech. Rep. 148, 1997.

[17] C. Le Goues, T. Nguyen, S. Forrest, and W. Weimer, "GenProg: A generic method for automatic software repair," *IEEE Transactions on Software Engineering*, vol. 38, no. 1, pp. 54–72, 2012.